



Fila



Prof. Rafael Mesquita
rgm@cin.ufpe.br

O que veremos nesta aula?

- Definição de Fila
- Implementação utilizando vetores
- Implementação utilizando Lista Encadeada
- Exemplos de uso

Definição de Fila

- Fila
- Estrutura de dados dinâmica
 - ▣ Inserção de elementos acontece sempre no final da fila
 - ▣ Remoção de elementos acontece sempre no início da fila
 - ▣ F.I.F.O.
 - *First in, first out!*
 - ▣ Primeiro elemento a entrar é o primeiro a sair
- Exemplos
 - ▣ Fila de impressão, fila de banco
 - ▣ Fila de processos de um sistema operacional na espera para execução

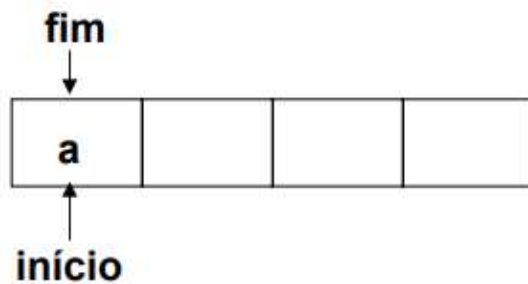
Definição de Fila

- Fila
- Operações básicas
 - Inserir
 - `enqueue(novo_elemento)`
 - Insere um novo fim da fila
 - Remover
 - `dequeue()`
 - Retira um elemento do início da fila
 - Pode apenas remover ou remover e retornar o elemento

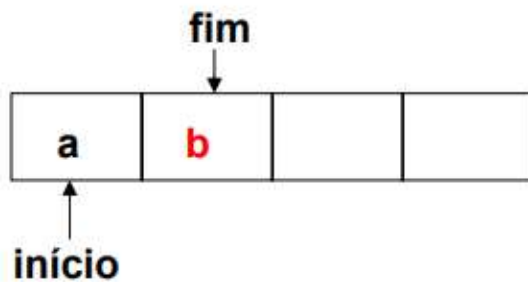
Definição de Pilha

□ Funcionamento

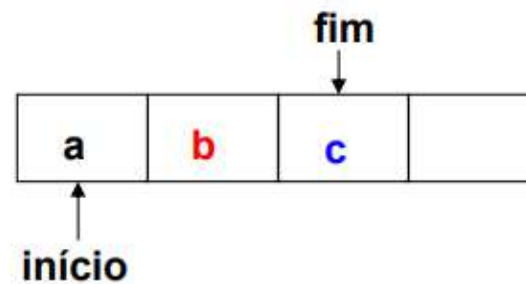
1- insere(a)



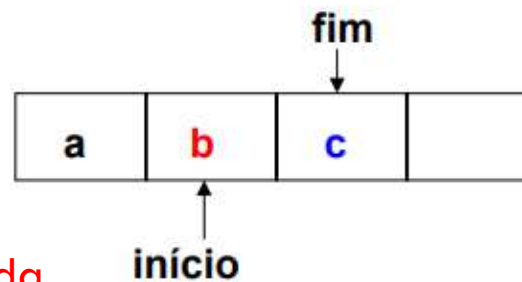
2- insere(b)



3- insere(c)



4- retira()

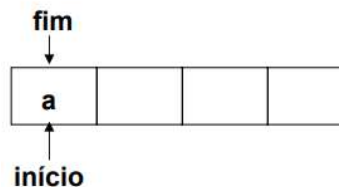


Caso implementação
dinâmica seja utilizada,
memória de 'a' deve
também ser liberada

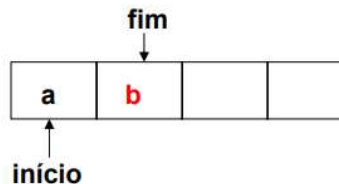
Implementação (vetor)

- Elemento é sempre inserido no índice “fim”
 - ▣ Fim anda uma posição (próxima posição livre)
- Remoção ocorre sempre no índice “início”
 - ▣ Início anda uma posição

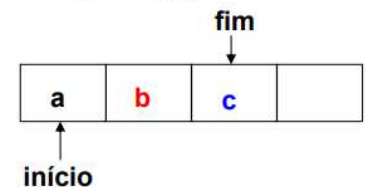
1- insere(a)



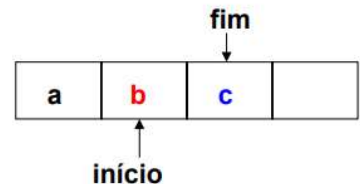
2- insere(b)



3- insere(c)



4- retira()



Implementação (vetor)

□ Problema

- Desperdício de espaço

- “fim” pode chegar à última posição e ainda existir espaço no início da fila

- Solução: estratégia circular

- Se a última posição estiver ocupada e existir espaço no início, próxima inserção ocorre no início

- $\text{fim} = (\text{início} + n) \% N$

- Início: posição inicial

- Fim: posição final

- n: número de elementos inseridos

- N: tamanho do vetor

Implementação (vetor)

□ Definição e criação

```
#define N 100
```

```
typedef struct fila {  
    int inicio;  
    int n_inseridos;  
    float vet[N];  
}Fila;
```

```
Fila* cria() {  
    Fila* fila = (Fila*)malloc(sizeof(Fila));  
    fila->n_inseridos = 0;  
    fila->inicio = 0;  
    return fila;  
}
```

Implementação (vetor)

□ Inserção

```
void insere(Fila* f, float valor)
{
    int fim;
    if (f->n_inseridos == N)
    {
        printf("capacidade máxima atingida! elemento nao inserido!");
        return;
    }
    fim = (f->inicio + f->n_inseridos) % N;
    f->vet[fim] = valor;
    f->n_inseridos++;
}
```

Implementação (vetor)

□ Utilitários

```
int fila_vazia(Fila* f)
{
    return f->n_inseridos == 0;
}
```

```
void libera(Fila* f)
{
    free(f);
}
```

```
void imprime(Fila* f)
{
    if (fila_vazia(f))
    {
        printf("fila vazia!");
        return;
    }
    printf("\n");
    for (int i = 0; i < f->n_inseridos; i++)
        printf("%f ", f->vet[(f->inicio+i)%N]);
}
```

Implementação (vetor)

□ Remoção

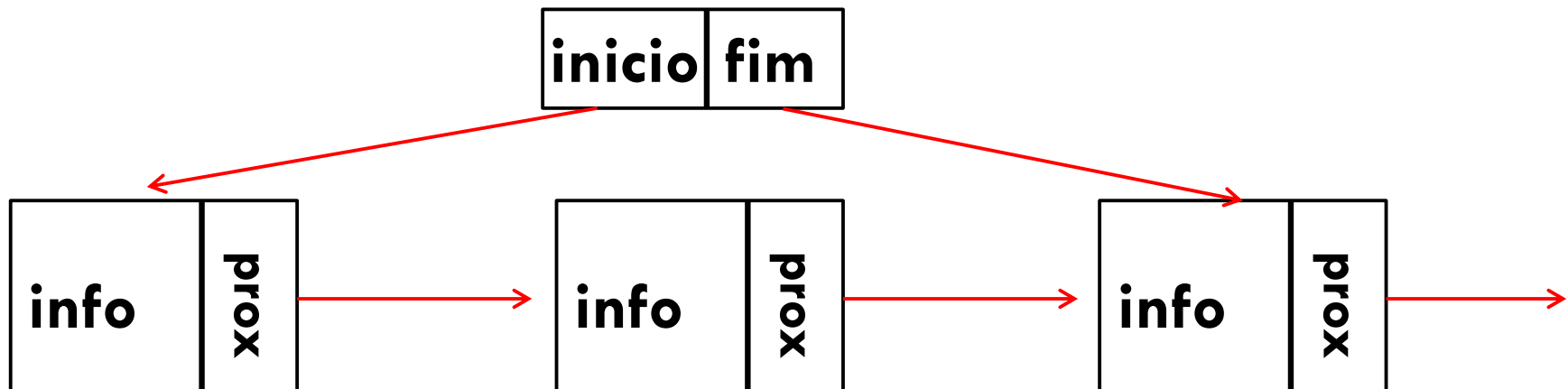
```
void remove(Fila* f)
{
    if (fila_vazia(f))
    {
        printf("fila vazia!");
        return;
    }

    if (f->inicio == N - 1)
        f->inicio = 0;
    else
        f->inicio++;
    f->n_inseridos--;
}
```

Implementação (Lista Encadeada)

```
typedef struct lista{  
    float info;  
    struct lista* prox;  
}Lista;
```

```
typedef struct fila {  
    Lista* inicio;  
    Lista* fim;  
}Fila;
```



Implementação (Lista Encadeada)

```
Fila* cria()
{
    Fila* fila = (Fila*)malloc(sizeof(Fila));
    fila->inicio = NULL;
    fila->fim = NULL;
    return fila;
}
```

```
int fila_vazia(Fila* f)
{
    if (f == NULL)
        return true;
    return (f->inicio == NULL);
}
```

Implementação (Lista Encadeada)

□ Inserção

```
void inserir(Fila* f, float valor)
{
    Lista* novo = (Lista*)malloc(sizeof(Lista));
    novo->info = valor;
    novo->prox = NULL;

    if (fila_vazia(f))
        f->inicio = novo;
    else
        f->fim->prox = novo;

    f->fim = novo;
}
```

Implementação (Lista Encadeada)

□ Remoção

```
void remover(Fila* f)
{
    Lista* aux;
    if (fila_vazia(f))
    {
        printf("\nRemoção não realizada. Lista vazia!");
        return;
    }
    aux = f->inicio;
    f->inicio = aux->prox;
    if (f->inicio == NULL)
        f->fim = NULL;
    free(aux);
}
```

Implementação (Lista Encadeada)

□ Liberando memória

```
void libera(Fila** f)
{
    if (fila_vazia(*f))
        return;
    Lista* var = (*f)->inicio;
    while (var != NULL)
    {
        Lista* aux = var->prox;
        free(var);
        var = aux;
    }
    free(*f);
    (*f) = NULL;
}
```

Implementação (Lista Encadeada)

□ Impressão

```
void imprime(Fila* f)
{
    if (fila_vazia(f))
    {
        printf("\nfila vazia!");
        return;
    }
    printf("\n");
    for (Lista* var = f->inicio; var != NULL; var = var->prox)
        printf("%f ", var->info);
}
```

Implementação (Lista Encadeada)

□ Testando código

```
int main()
{
    Fila* f = cria();
    inserir(f, 2.0);
    inserir(f, 2.3);
    inserir(f, 2.8);
    imprime(f);
    //2 2.3 2.8
    remover(f);
    imprime(f);
    //2.3 2.8
    libera(&f);
    imprime(f);
    //fila vazia
    f = cria();
    inserir(f, 2.9);
    inserir(f, 2.10);
    inserir(f, 2.13);
    imprime(f);
    //2.9 2.10 2.13
    remover(f);
    inserir(f, 2.14);
    remover(f);
    imprime(f);
    //2.13 2.14
    libera(&f);
    getchar();
    return 0;
}
```

Exercícios

1. Crie sua própria implementação de Fila utilizando uma lista encadeada com as funcionalidades apresentadas nesta aula
2. Escreva um programa em C para mostrar como uma fila pode ser implementada por meio de duas pilhas.

